EAST WEST UNIVERSITY
Department of Computer Science and Engineering
B.Sc. in Computer Science and Engineering Program
Midterm Assessment Project, Fall 2025 Semester

Course:	**CSE407 Green Computing, Section: 02**
Student ID:	2022-1-60-265
Student Name:	Rifa Azad

Project Title:

Gmal Energy Monitoring – a browser extension to calculate carbon footprint

Checklist:

- ✓ Energy consumption monitoring has been done for at least a week.
- ✓ I have prepared an energy monitoring dashboard/webpage with detailed and summarized information.
- ✓ I have considered the financial and business aspects.
- ✓ I did a PESTLE analysis on the project.

Declaration

I declare that the project was done by me, and the submitted information is free from any unfair means. I complied with all the relevant legal and organizational requirements.

Signature

Name: Rifa Azad

Date: 7 November 2025

Table of Contents

1. Executive Summary 3

2. Brief description of the work 3

3. Methodology of work 3

4. Detailed description of each of the steps 4

1) Planning *4*

2) Researching *4*

3) Purchasing *5*

4) Configuration *5*

5) Further processing *5*

6) PESTLE Analysis: *6*

5. Challenges and Solutions 7

6. Demonstration 8

a) A video demonstration of the working of the extension has been submitted alongside. *8*

b) Pictures of the extension *8*

7. Discussion on issues: 11

a) Checklist (After Appendices) *11*

b) Address EP1 & EP2 ..... *11*

8. Appendices 12

a) Codes : https://github.com/rifa-zd/Gmail-CO2e-Energy-Monitoring---broswer-extension *12*

References 13

1. Executive Summary

The project is purely software-based energy monitoring system for tracking the carbon footprint and energy use from Gmail activities since logging in time. The primary object of this system is to use Gmail API data to monitor metrics such as emails sent/received, attachments, and mailbox size to calculate CO2 emissions *(g)* and energy *(watt-hours)* based on real-world metrics.

The extension has a user interface with tabs for overview, mailbox stats, tips, and formulas. It shows today's activity along with a line graph, weekly trends, and cumulative charts using `Chart.js`. Data is stored locally in browser's extension storage and cleaned up after 15 days. The system uses weekly emission statistics with real-world equivalences like phone charges or car driving distances to provide users a vision of how much energy their mailbox is consuming and affecting the environment. Also, based on user usage personalized tips are suggested to reduce their environmental impact. The system supports exporting data as CSV for further analysis.

Metrics and formulars used in the system are based on recent works *(details discussion in section 4.2)*. As the system used local storage to store data, the main challenge of the project was to handle the system in a way that it does not hit Gmail API limit or fill up browser's storage. It was planned to make a web application to get rid of the storage problem by using cloud service but due to issues like Firestore calls, Gmail Api not responding with the Firebase call, the web application was put on hold.

Overall, the extension was created to monitor user Gmail activity to raise awareness that digital life has a real environmental cost. Even though they seem like a small portion, it is found that Global emails emit about 150 million tons CO2 yearly, or 0.3% of world emissions using the range put by Mike Berners-Lee in 'How Bad are Bananas?' book which has been widely in many articles related to email emission [1] along with mention in Carbon Literacy Project (2024) [2] .

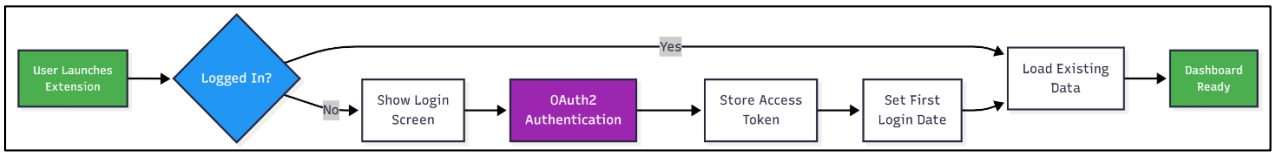
2. Brief description of the work

The Gmail energy monitor extension is a tool that calculates the energy consumption of emails in the inbox. Upon refresh, it will sync with the mailbox (sent, received mail with or without attachments). Based on mail stats, the system will calculate energy consumption and measure carbon emissions. The system will show charts – one for the day-24-hour (over the hours), another weekly (over 15 days, each 15-day-old data will be deleted to preserve browser storage), and the cumulative charts an aggregated result of the carbon emission of the user email habit. Also, it will show carbon emissions per day, average/day, mailbox stats, along with optimization tips based on the energy monitored.

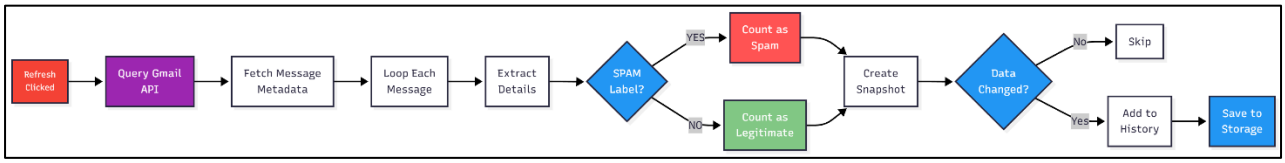
3. Methodology of work

Below is the systematic workflow of the Gmail Energy Monitoring System:

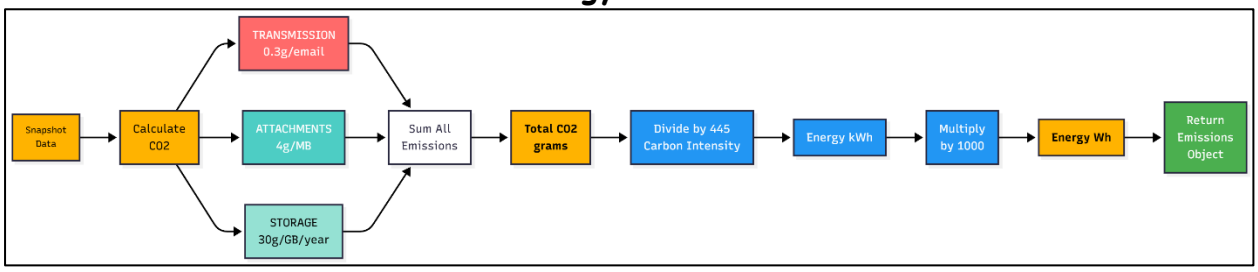
User Authentication Flow



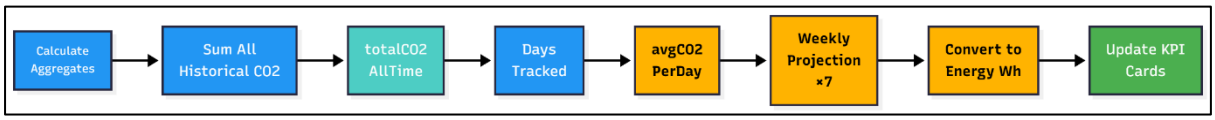
Gmail Data Fetch & Process



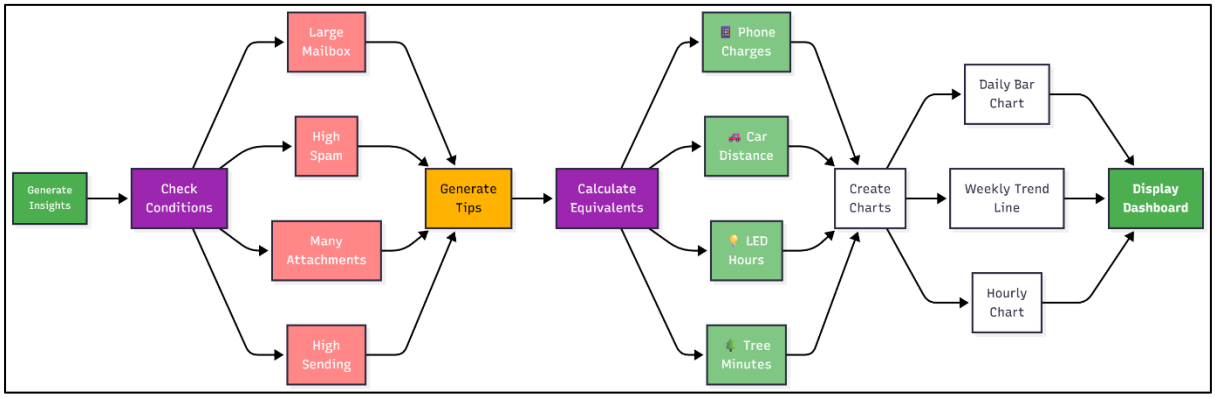
CO2 & Energy Calculations



Data Calculation & UI Flow



UI, Metrics & Visualizations



4. Detailed description of each of the steps

1) Planning

- The initial plan was to build both a web application and a browser extension but was only able to complete the browser extension system, which will use the browser's user account to access their Gmail API from where they will fetch the Gmail data.
- Planned Features: Login, fetch data, calculate CO2/energy, show dashboards (carbon & energy metrics based on daily and weekly trends, along with charts, a cumulative chart that will be shown when the pop-up opens at first), provide tips to minimize carbon emission and reduce energy consumption by showing how their usage are equivalent to different real life carbon emission metrics.
- The goal was to use the Gmail Api for real user data. The system will start working the moment the user logs in then will keep tracking data unless user removes the browser extension rather than using whole Gmail data.

2) Researching

To use metrics that will calculate gmail emissions and energy some research work was done to find the best practices to ensure that the extension align with the most recent practices. Among the work related to email carbon footprint, Mike Berners-Lee 'How Bad are Bananas?' book is the most citied work, almost every research on this matter moves back to this analysis. The average carbon footprint of an email/spam is 0.3g CO2e [3] while a standard one is 4g CO2e [2] whereas an email with an attachment can go 4.2g to 50g CO2 per MB [4].

To analyses the footprint of data storage where emails are stored, article [5] stated that it

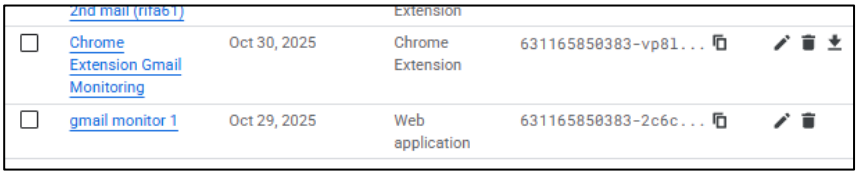
requires about 0.1 kWh electrical energy per GB per year for 30g CO2e. IEA Electricity 2025 report stated that in global range of email energy is around 445g CO2/kWh. [6].
The equivalence was found in greenhouse gas equivalencing calculator to find the value of phone charge equivalent of 8g CO2e full charge [7] . LED bulb power rate with 4.45g CO2e was mentioned in IEA 2024 emissions factors [8]. The car driving equivalent was sourced in [9] whereas the tree carbon absorption was mentioned in [10] to find how much carbon is absorbed by trees.

3) Purchasing

No, it is a purely software system.

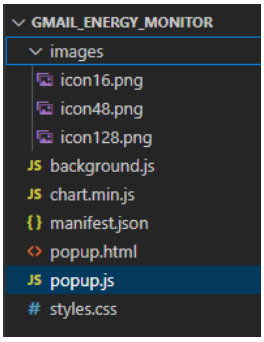
4) Configuration

- `Manifest.json` was configured, a must for browser extensions, for permissions including identity, storage, and Gmail API read only access, also in this file the *client id* from google cloud *OAuth2* credential were set up.



- While creating the credentials it is necessary to provide *item id* for browser extension, which is found in the browser upon uploading the code folder in extension section in the developer's mode.
- The system was developed with HTML, CSS, and JavaScript (vanilla) and Chart.js.

- In case of development, test users are required to be set who will test the system that is using Gmail API, therefore 3 test users were given in the cloud project. Only the test users can log in to the extension, any other mail will cause error and will not authenticate. But this is not a major issue as when the system will be in production, after Google verification, the browser could be found in chrome web store and anyone with google account will be able to use it, which a future scope for this project.
- A `background.js` is also a must from extension folder, where the background tasks are instructed. In terms of gmail energy monitoring too, it holds the logic for data refresh, cleanup 15 days old data, auto refresh was written too ***but due to hitting the Gmail API limit, this portion of the code was commented out.***
- All the backend logic was in `popup.js` file where the front end was manipulated using `popup.html` and `styles.css`..
- The image folder contains images of 16px, 48px, 128px that are used by the extension.



5) Further processing

- After the configurations were set, the first logic written was the Gmail Api access (authorization), and fetch Gmail data and store them in the browser local storage (extension storage).
- Following that, the logic calculation and chart generation were configured. At the initial stage, only *avg CO2/day* and *weekly energy,* along with the weekly trend, were monitored, along with a cumulative chart that aggregates emissions of each day. Later, daily charts were added for better monitoring along with metrics (total mail per day, CO2 of that day, with energy & data transferred).
- To know whether mail fetching was accurately done, a mailbox tab was added that will keep tabs of mail activity of the Gmail inbox since the login day to show over all statistics.
- Formula sections were added to let the user know which metrics are being used for calculations,

along with a “export CSV” which will let user download all information since logged in day till today.

- To sync the extension with user inbox, a refresh button was added, upon clicking it will refresh the statistics to match with user’s Gmail.
- Some core logic functions used in `popup.js` are:
    - Event listeners were set that will be trigger once buttons (login, tabs, refresh, export) are clicked to show their content in the below empty area by making the UI interactive.
    - Google require access token, a short lived key that will act as a temporary password to let the extension securely access Gmail data via Google's API- this work is done by **handleLogin()** by also saving first login date, which is later used by **fetchGmailData()** for fetching data and keep track of mailbox statistics (attachments and spam).
    - Helper functions were used to count the number of attachments & theirs sizes, for conversion of CO2 to kWh/Wh and a calculation function was used for the core logic calculation.
    - After fetching, data were stored in chrome local storage and later used by u**pdateAllStats()** to calculate totals like average daily CO2, updates all screen text (stats, mailbox, spam), today’s section, along with the calculating values that are needed for the charts and used to generate tips based on user stats.
    - **exportCSV()**: Grabs all history, adds calculated CO2/energy per snapshot to a CSV string, includes a summary of metrics, then downloads it as a file.
    - For calculations, mail used were sent/received, spam, attachment used in them. Unread mails do not affect emission calculation not even deleted mails as when the mail arrived in the mailbox, energy and emissions were calculated then, and the temporary storage they used by staying in the “trash” folder are negligible.
- The chart for daily trend along with its metrics (total mail per day, CO2 of that day, with energy & data transferred) would start from midnight till the end of the day showing a full day monitoring of gmail energy usage and carbon emissions.
- But the weekly trend, cumulative chart, mailbox statics, tips for optimization and the equivalent tab are calculated based on the day first logged and will continue till user deletes the extension manually as there is no log out option in the extension (to avoid losing data).
- For the optimization tips, simple if-else conditions were used based on user usage, carbon emissions based on their weekly trend, as it’s a small-scale project (with gmail messages reaching 500 will cause warning and urge to refresh storage to avoid Gmail API limit hit, as no cloud was used). Optimization are used for large scale complex systems where optimal solution is core. Besides, for email energy monitoring, it uses simple fetch, lop, and calculation that does not require extensive optimization algorithms.

6) PESTLE Analysis:

- Political – As this system is monitoring the energy uses of gmail, detecting carbon emission of digital technology, part of green environment, a political party can use this a part of their campaign to reach people by assuring that they will be able to make the environment greener not physically but technologically too.
- Economic – As this tool is a sustainable extension, with proper marketing this could a great product for tech giants to promote their sustainability effort which are often found faulty, promoting their business with a eco-tool.
- Social – As people awareness to sustainability is increasing, a gmail energy tracking app, will help user track their daily email habits, making them more aware toward carbon emissions.
- Technological – Even though no IoT tools were used but the infrastructure of the system uses Gmail API and Chrome storage, therefore the rapid evolution in these sectors could bring a much more reliable monitoring system for the Gmail.
- Legal – The system uses the Gmail data in read-only mode, meaning only TO, FROM, TIME STAMP not the body of the email or content of attachment sent. Therefore, once the system is deployed with Google verification it will abode by the legal rules of

technological field, causing users no worries.

- Environmental – The whole system was created by keeping the environment in mind, to find the carbon emission of a digital field that effect other metrics (data centers, cloud) energy consumption.

5. Challenges and Solutions

Extension:

1. On first try, after closing the browser all data were wiped from the local storage due to access token expiration and Gmail API limit hit. As the code was structured in a way that every time the popup opens and after every 30 minutes the extension will auto refresh to fetch latest emails that cause the API to hit limit and browser storage was wiped out.
   - ✓ To avoid this issue the code for auto refresh was commented out (to use in future) and a manual refresh button was added which added a fault in the system that if any mail activity was deleted in between refresh then that data will not be in the monitoring system.

2. Even though the refresh problem was fixed but the data was still not being fetched properly as the OAuth callback and token management was not properly managed that caused the initial system to lose access tokens then when the refresh button was clicked to sync with Gmail causing the system unable to update data.
3. **Faced Content Security Policy (CSP)** violation in Chrome extensions where the chrome extensions don't allow external script sources in CSP for security reasons.
   - ✓ Therefore, instead of using the CDN link in the script tag, used the `chart.min.js` file locally.

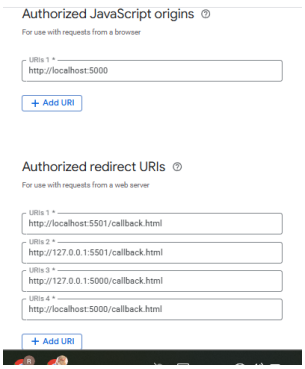
4. The energy calculation used `toFixed(2)` that made small values like `00.337 Wh` (energy, conversion from CO2) to become `0.0,` which took quite a while to find and fix as the calculation logic was correct but display function was wrong.

5. The initial logic behind `avg CO2/day & weekly energy` was written to be changed upon any mail activity, which is not correct, as they should be updated based on daily emission monitoring, therefore, their calculation logic was changed.

6. To make sure the extension runs smoothly, the user must be registered as a test user under the project in Google Cloud, and the *item id* found in the browser folder detail must be given in the credentials, or else that user will not be able to use the extension as the system is still in the development stage.

For the browser extension, most of the issues were about UI and the calculation logic that is in depth in the code, identifying them required considerable effort, but they were not as critical as the problem faced with the web application.

- ✗ To enable credentials for web application, "***Redirected URI***" (picture on side) are required to provide that will be allow user to return to main web application (the provided link) after Google successfully verified user (authentication). But in my case, this URI became the core reason why I was unable to develop the web application despite having the front end and backend logic working correctly (checked the UI as a normal web application).



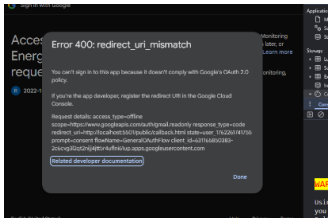
- ✗ At one stage, I was able to successfully log in using gmail in the web application but could not see any update which I thought could be the reason of the logic that fetch data after log in time (picture given below). But after closing the browser, again when trying to check the updates after a while, the whole application collapsed, and could not even logged in. The error was "Redirect URI

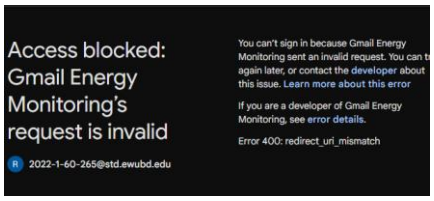
mismatch" even though not a single line of code was touched or the URI in the Google cloud were changed.

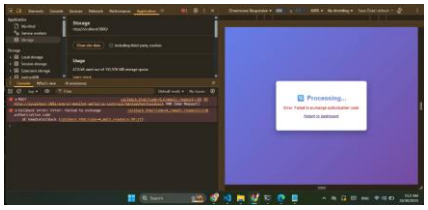
- The possible reason for this could be the session end but even from verifying from the console log error messages, could not fix the problem. After multiple try, re-starting the web application from scratch twice was not able to find the solution for the problem, keeping the code incomplete.

- CORS (Cross-Origin Resource Sharing) Issue: When the cloud function tried to fetch Gmail data it failed due to CORS, a security protocol of the browser that controls for web applications interaction with other domains – another reason why the gmail data was not visible in the web application on successful log in once.

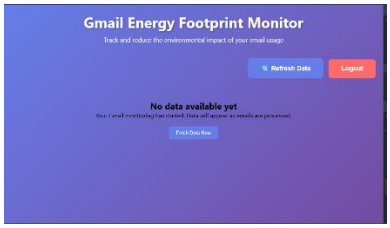
× At some point, even the Firebase emulator that let the code run locally using localhost broke, and I could not locate the cause. Some snapshots of the problems are below:

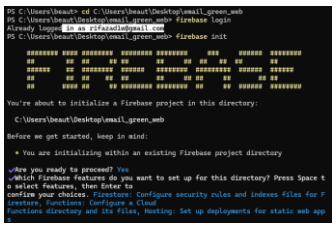


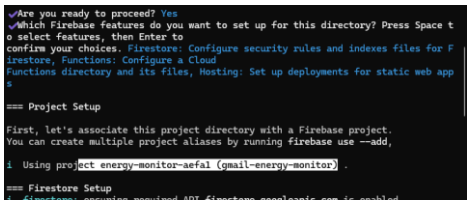




✓ Even though the actual application could not run later on but on the very first try it ran, some snapshot for that are:







Despite a failed application, it's a part of the project because a great amount of time was invested to identifying its problems, tracing their sources, and attempting to resolve them.

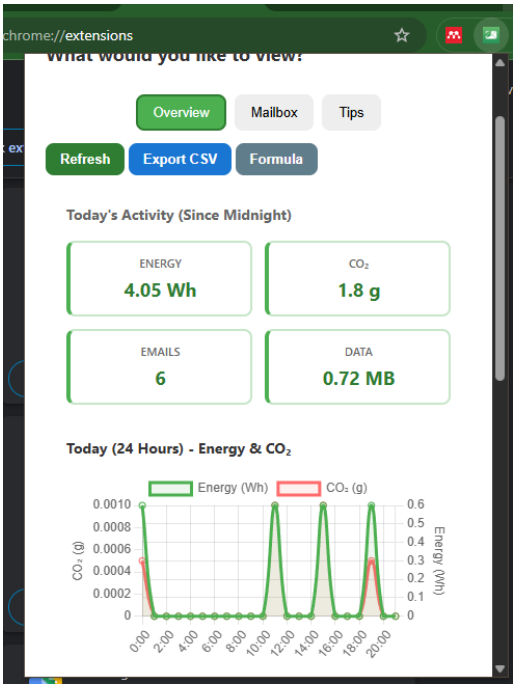
6. Demonstration

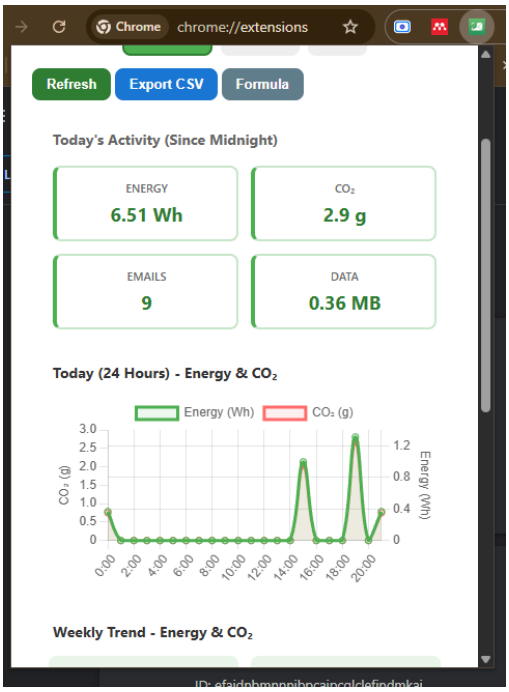
a) A video demonstration of the working of the extension has been submitted alongside.
b) Pictures of the extension

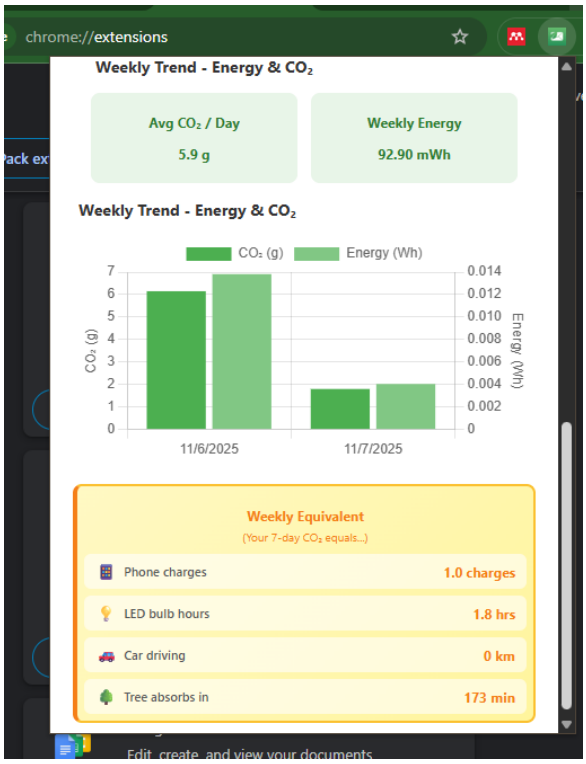
Dashboard

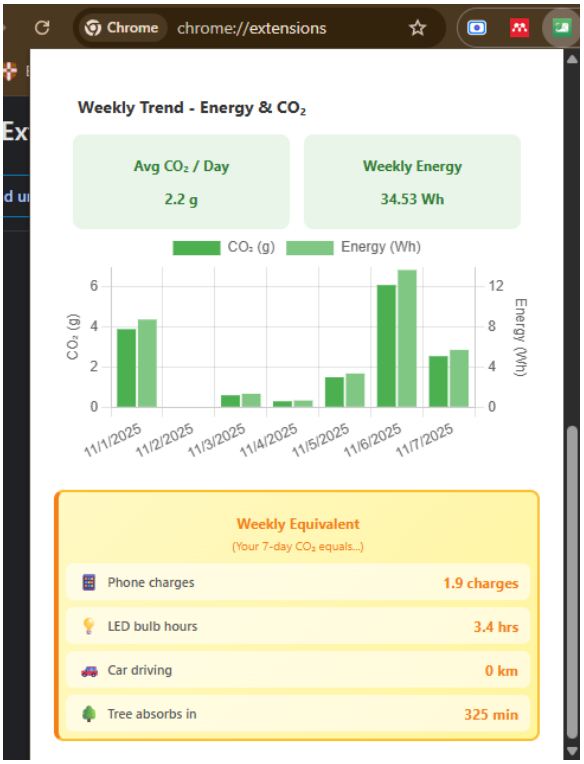
- Monitoring of two different emails – one had 7-day monitoring, another (a lot of subscription mails) with 2 days

Daily & Weekly trend

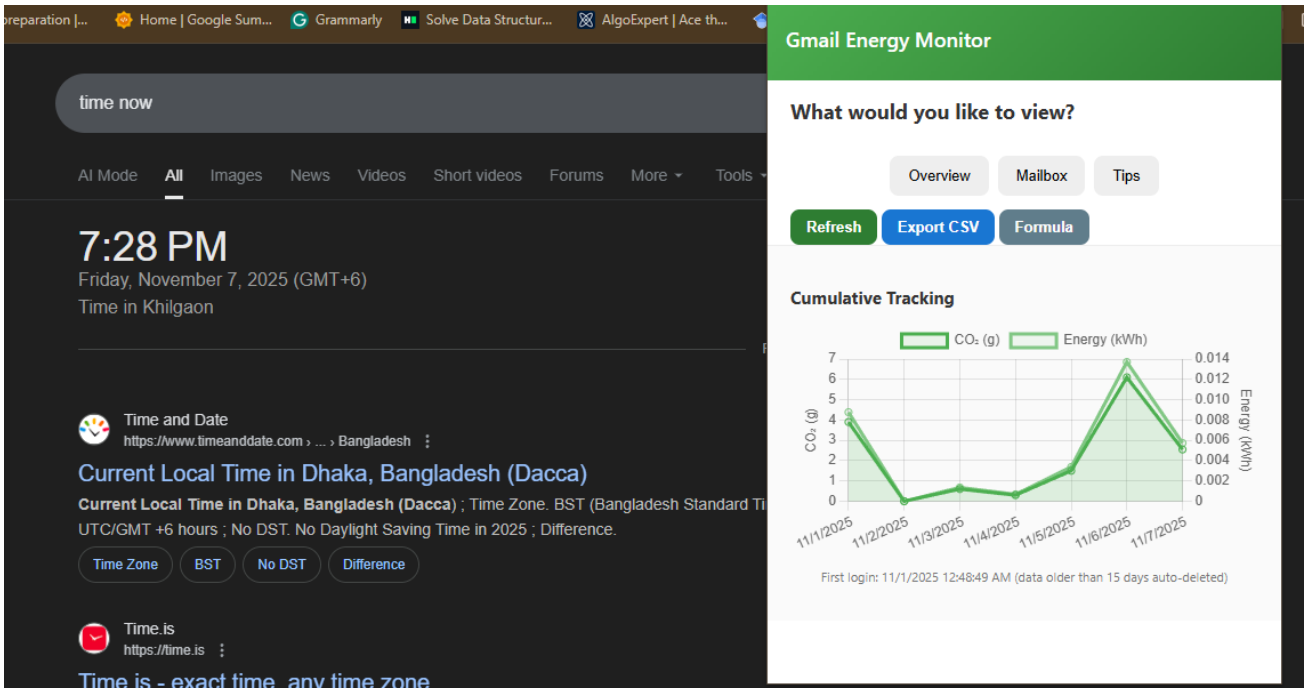




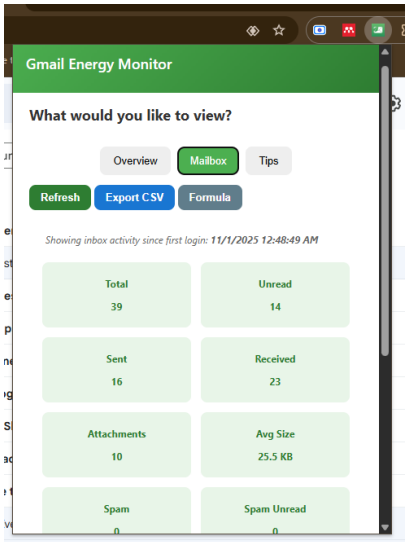


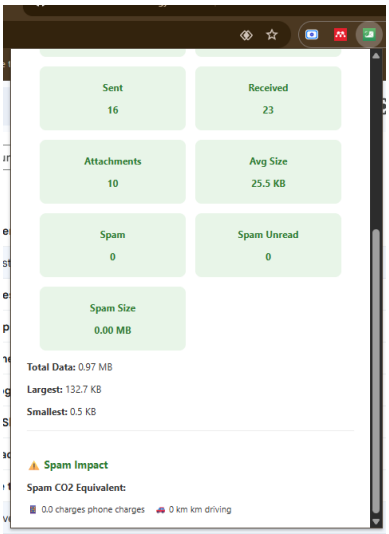


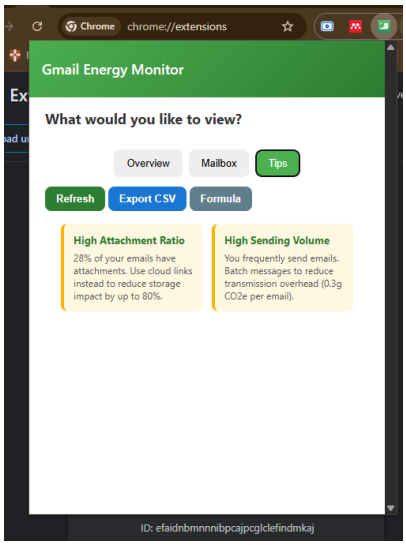
Seven-day Monitoring of Gmail Activity monitoring

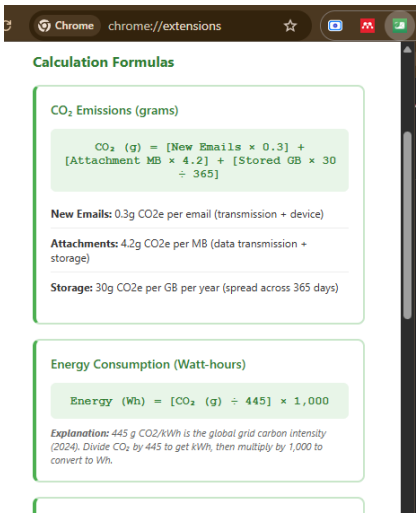


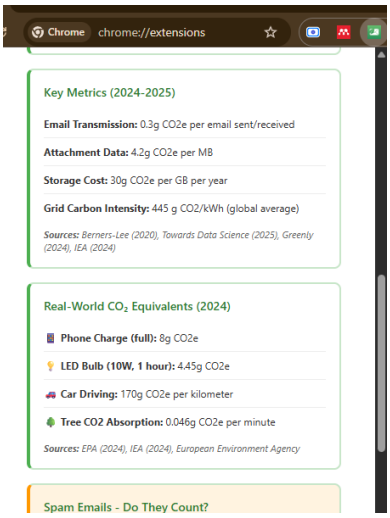
Other Statistics , Tips for Optimization & Formula Tab

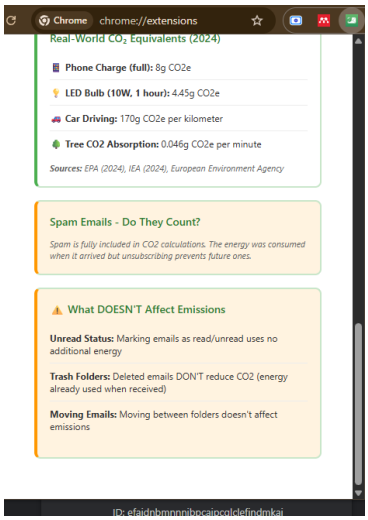












7. Discussion on issues:

a) Checklist (After Appendices)
b) Address EP1 & EP2

EP1: Depth of Knowledge

• In-depth engineering knowledge

The Gmail Energy Monitor Chrome Extension applies core software engineering skills - API integration, and data processing, cloud function handling (even though it was not able to complete but attempted). Engineering principles from web development are applied along with Gmail API for read-only access that require the understanding of OAuth 2.0 authentication flows.

Also, the metrics used for calculations, compare were derived from real life energy monitoring metrics for sustainability in the tech world.

• First principles approach

The system fetched gmail data and broke them down in number of mails sent, received, spam mail, attachment sizes on per mail; breaking down the problem to fundamental components that align with first principal approach.

• Analyze & evaluate solution

The analytics in the system were based on user usage and evaluate against most acceptable terms(cited in references) that raise awareness among users. The system has user activity base solution rather than generic solutions.

EP2: Conflicting Requirements

• Wide-ranging technical issues

During development, several issues appeared: Gmail API limit, storage limit, manual refresh rather than automatic.

As the extension was developed based on Gmail activity, compatibility with other browser comes in questions.

The failure in developing web application remains.

• Trade-offs & compromises

In the calculation, global metrics were used, ignoring regional values – a compromise in monitoring tool.

By not using cloud, keeping the data in local storage reduce complexity at the cost of privacy.

Simple, direct formulas instead of complex optimizations brought benefit for this small scaled, single user (in development stage, only registered test user can access) extensions.

To avoid hitting API limit, auto refresh was compromised to manual refresh.

• **Compare & assess conflicts**

Conflicts generated while developing the system were assessed as best practices. Design choices were deeply affected by user choice to provide a clean, transparent system. Therefore, it can be said that conflicts are resolved to provide user a eco-tool with efficiency prioritized.

8. Appendices

a) Codes :

https://github.com/rifa-zd/Gmail-CO2e-Energy-Monitoring---broswer-extension

Web Application:
https://github.com/rifa-zd/Gmail-CO2e-energy-Monitoring---web-application

Checklists:

Checklist of Issues	**Remarks (if any)**
*Planning and researching?	Yes
Data Collected?	Yes, from Gmail
Data Stored?	Yes, in chrome storage
Data displayed in the dashboard?	yes
*Real-time or stored data?	Real time then stored in chrome local storage
*Wattage of the chosen device?	N/A
*Wattage of the measuring equipment?	N/A
*AC Power: Why/not apparent power? Why/not Instantaneous Power? AC power vs. DC power?	N/A
*Documentation	Yes
*Safety: Electrical Insulation and Isolation?	N/A
*Caution with overclocking/flushing	N/A
*API Issues: Did you get it? How? If not, how did you solve this problem?	Yes, refresh token problem

*UI/UX issues? Standard components of an energy dashboard?	Issue faced and solved. Yes
*User Manual?	Yes (formula tab)
*Future Extensions and Limitations?	Web application would be a greater full scale product
*Installation, Operation and Maintenance?	Require extension knowledge which
*Recurring costs	None
*Cost Accounting?	Free
*Business Aspects? Cost savings and ROI? Value of this product/service? Justification?	Could be good marketing product to promote sustainability in digital field.
*Reliability? Never failed? Any fail-safe mechanisms?	Extension never failed, just some UI problem faced.
*Accuracy? Calibration?	N/A
*Data quality? Sampling rate? Crosstalk and interference? Accuracy and calibration?	Checked with inbox mail statics and data mismatch was not found
*Scalability?	Storage could be a issue.
*Interoperability?	Followed standard filing system
*Data Security? Important or not in this case?	Important
*Compliant with regulations?	Yes
*Environmental Impacts? PESTLE analysis?	Yes

References

[1] E. Matcher. [Online]. Available: https://www.ecomatcher.com/the-climate-cost-of-internet-data/.

[2] C. L. Project. [Online]. Available: https://carbonliteracy.com/the-carbon-cost-of-an-email/.

[3] Pawprint. [Online]. Available: https://www.pawprint.eco/eco-blog/carbon-footprint-email.

[4] [Online]. Available: https://towardsdatascience.com/how-to-decrease-the-carbon-footprint-of-digital-communication-f3186673818c/.

[5] greenly. [Online]. Available: https://greenly.earth/en-gb/blog/industries/what-is-the-carbon-footprint-of-data-storage.

[6] I. report. [Online]. Available: https://www.iea.org/reports/electricity-2025/emissions.

[7] p. epea. [Online]. Available: https://www.epa.gov/energy/greenhouse-gas-equivalencies-calculator.

[8] e. factor. [Online]. Available: https://www.iea.org/data-and-statistics/data-product/emissions-factors-2024.

[9] car. [Online]. Available: https://ourworldindata.org/travel-carbon-footprint.

[10] tree. [Online]. Available: https://www.thetreeapp.org/blog/2024-12-20-how-much-co2-does-one-tree-absorb/.